

Documentacion Tecnica

Microservicio: checking-service (Motor de Reservas)

Sistema de Gestion de Co-working

Universidad Jose Antonio Paez

Escuela de Computacion

Ingenieria de Computacion | 7mo Semestre

Integrante	Leonardo Correa - C.I. 30889380
Lenguaje / Framework	Rust 1.x + Axum 0.8 (SQLx + PostgreSQL)
Puerto del servicio	:8001
Fecha de entrega	03 de junio de 2026
Repositorio	github.com/lpzzlnrd/Coworking
Ruta del servicio	services/checking-service

1.1 Responsabilidad del servicio

El checking-service es el motor de reservas del sistema de co-working. Su unica responsabilidad es gestionar el ciclo de vida completo de una reserva de espacio: creacion, consulta, confirmacion y cancelacion, garantizando que un mismo espacio nunca quede reservado por dos personas en horarios que se solapan. Mantiene ademas una bitacora de auditoria con cada cambio de estado de la reserva.

Lo que NO hace marca sus limites: no autentica usuarios ni emite tokens (eso es del auth-service), no administra el catalogo de espacios ni sus precios (eso es del spaces-service), y no procesa pagos ni facturas. Solo consume la identidad ya verificada del usuario y referencia los espacios por su identificador.

1.2 Por que existe como microservicio separado

La deteccion de solapamientos es una regla de negocio critica que exige una garantia de concurrencia fuerte: dos peticiones simultaneas para el mismo espacio y horario no pueden ambas tener exito. Aislar esta logica en un servicio dedicado permite que la restriccion de exclusion viva junto a su tabla, en una unica base de datos donde PostgreSQL arbitra la condicion de carrera de forma atomica.

Si esta logica viviera dentro del spaces-service o del auth-service, cada cambio en las reglas de reserva obligaria a redespregar un servicio con otras responsabilidades, y la tabla de reservas competiria por recursos con consultas de catalogo o de login. Separarlo permite escalarlo y desplegarlo de forma independiente segun su propia carga.

1.3 Integracion con otros servicios

El servicio se comunica con dos vecinos. Con el auth-service de forma implicita: cada peticion protegida trae un JWT firmado por auth-service, que checking-service verifica localmente con el secreto compartido HS256, sin hacer una llamada de red por peticion. De forma opcional, al crear una reserva consulta por HTTP al auth-service para confirmar que el usuario aun existe. Con el spaces-service intercambia unicamente el space_id (UUID) que viaja en el cuerpo de la peticion de creacion; checking-service confia en ese identificador y registra la reserva contra el.

2.1 Endpoints del servicio

Metodo	URL	Descripcion	Token	Rol
GET	/health	Health check del servicio y la base de datos	No	Publico
POST	/reservations	Crear una nueva reserva	Si	Cualquiera
GET	/reservations	Listar las reservas del usuario autenticado	Si	Cualquiera
GET	/reservations/{id}	Obtener una reserva por su identificador	Si	Cualquiera
PATCH	/reservations/{id}/confirm	Confirmar una reserva en estado pending	Si	Dueno
DELETE	/reservations/{id}	Cancelar una reserva	Si	Dueno

Nota de autorizacion: 'Dueno' significa que solo el usuario que creo la reserva (su user_id coincide con el sub del JWT) puede confirmarla o cancelarla; la regla se valida en la capa de aplicacion (funcion ensure_owner).

2.2 Ejemplo completo - request y response del endpoint principal

Metodo y URL del endpoint elegido:

```
POST /reservations
Host: localhost:8001
Authorization: Bearer <jwt-emitido-por-auth-service>
Content-Type: application/json
```

Request body:

```
{
  "space_id": "a1b2c3d4-0000-0000-0000-000000000001",
  "start": "2026-06-10T09:00:00Z",
  "end": "2026-06-10T11:00:00Z",
  "notes": "Reunion de equipo de producto"
}
```

Response (HTTP 201 Created):

```
{
  "id": "f7e6d5c4-1111-2222-3333-444444444444",
  "user_id": "b0b0b0b0-0000-0000-0000-000000000001",
  "space_id": "a1b2c3d4-0000-0000-0000-000000000001",
  "start": "2026-06-10T09:00:00Z",
  "end": "2026-06-10T11:00:00Z",
}
```

```
"status": "pending",
"notes": "Reunion de equipo de producto",
"cancellation_reason": null,
"cancelled_at": null,
"created_at": "2026-06-02T14:30:00Z",
"updated_at": "2026-06-02T14:30:00Z"
}
```

Response de conflicto (HTTP 409 Conflict) si el horario se solapa:

```
{
  "error": "el espacio ya esta reservado en ese horario"
}
```

3.1 Nombre y descripción del algoritmo

Algoritmo: detección de solapamiento de intervalos mediante índice GiST con restricción de exclusión (EXCLUDE USING gist). Vive en la migración `migrations/20260524210758_initial_schema.sql` (restricción `reservations_no_overlap`) y se apoya en el `value object TimeRange` de `src/domain/value_objects/time_range.rs`.

Problema que resuelve: dado un espacio y un rango de tiempo solicitado [inicio, fin), determinar si ya existe alguna reserva activa (pending o confirmed) para ese mismo espacio cuyo rango se cruce con el solicitado. Si existe, la reserva se rechaza. La garantía es atómica: PostgreSQL evalúa la restricción dentro de la transacción del INSERT, por lo que dos peticiones concurrentes nunca pueden insertar ambas.

3.2 Como funciona paso a paso

Modelo de intervalos semiabiertos [inicio, fin): el fin es exclusivo, de modo que una reserva que termina a las 10:00 y otra que empieza a las 10:00 NO se consideran solapadas. Dos rangos A y B se solapan si y solo si $A.inicio < B.fin$ Y $B.inicio < A.fin$.

Pasos de una creación de reserva:

- 1. La capa de dominio valida que $inicio < fin$ (`TimeRange::new`). Falla barato, sin tocar red ni DB.
- 2. Se consulta opcionalmente al auth-service que el usuario exista.
- 3. Se ejecuta el INSERT con el rango convertido a `TSTZRANGE` de PostgreSQL.
- 4. Antes de confirmar la fila, PostgreSQL consulta el índice GiST sobre `time_range` buscando, solo entre las reservas del mismo `space_id` y en estado activo, si algún rango se cruza con el nuevo (operador `&&`).
- 5. El índice GiST organiza los rangos en un árbol balanceado, por lo que la búsqueda descarta ramas enteras y no recorre todas las filas. Encuentra un cruce en $O(\log n)$.
- 6. Si hay cruce, PostgreSQL aborta con el `SQLSTATE 23P01`, que el repositorio traduce a `DomainError::Overlapping` y el API responde 409. Si no hay cruce, la reserva se persiste.

3.3 Comparación de complejidad $O()$

	Nuestro algoritmo	Alternativa naive
Nombre	Índice GiST + EXCLUDE (&&)	Escaneo secuencial (for simple)
Complejidad	$O(\log n)$ por inserción	$O(n)$ por inserción
Con 100 reservas hace	~7 comparaciones ($\log_2 100$)	100 comparaciones
Con 10,000 reservas hace	~14 comparaciones ($\log_2 10000$)	10,000 comparaciones

3.4 Ejemplo numérico con datos reales del sistema

Supongamos la sala de reuniones 'Sala Roble' (`space_id a1b2c3d4-0000-0000-0000-000000000001`) que ya tiene 4 reservas confirmadas el 10 de junio de 2026:

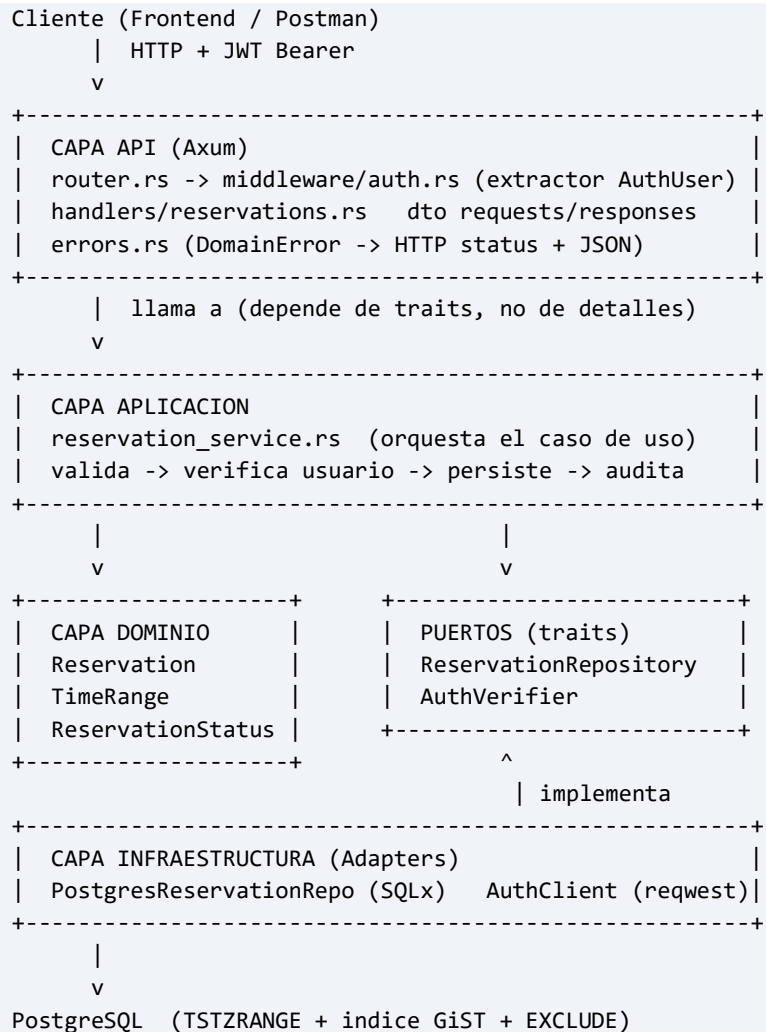
Reserva	Inicio	Fin
R1	08:00	09:00
R2	09:00	11:00
R3	13:00	14:30
R4	16:00	17:00

Llega una petición para reservar [09:30, 10:30). Con el escaneo naïve el servicio compararía el nuevo rango contra las 4 reservas una por una hasta hallar el choque con R2: 4 comparaciones en este caso, n comparaciones en el peor caso. Con el índice GiST, PostgreSQL navega el árbol de rangos del `space_id` y localiza el cruce con R2 en aproximadamente $\log_2(4) = 2$ comparaciones, devolviendo 409 Conflict.

El contraste se vuelve dramático con volumen real. Si 'Sala Roble' acumulara 10,000 reservas históricas y activas, el escaneo naïve haría hasta 10,000 comparaciones por cada intento de reserva, mientras que el índice GiST resuelve en unas 14. A escala de cientos de espacios con miles de reservas, esa diferencia es lo que separa una respuesta de milisegundos de un servicio que se cae bajo carga.

4.1 Diagrama de arquitectura del servicio

El servicio sigue arquitectura hexagonal (Ports and Adapters). Una petición fluye de afuera hacia adentro y la respuesta regresa por el mismo camino:



4.2 Por que elegimos este lenguaje para este servicio

Se eligio Rust con Axum porque la responsabilidad del servicio es alta concurrencia con correccion estricta. El motor de reservas debe manejar muchas peticiones simultaneas que compiten por los mismos espacios; el modelo de ownership de Rust elimina condiciones de carrera en memoria en tiempo de compilacion, sin recolector de basura que introduzca pausas impredecibles bajo carga.

Ademas, el sistema de tipos permite modelar las reglas de negocio de forma que los estados invalidos no compilen: un TimeRange no puede existir con inicio mayor o igual al fin, y un Result obliga a manejar el caso de solapamiento explicitamente. Para un servicio donde un error de

conurrencia significa una doble reserva, esa garantía del compilador es el argumento decisivo frente a lenguajes mas permisivos.

4.3 Como funciona la verificación JWT en este servicio

- El token llega en el header Authorization con el formato 'Bearer <token>'.
- El extractor AuthUser (src/api/middleware/auth.rs) intercepta cualquier handler que lo declare en su firma; si falta el header o el formato, responde 401.
- decode_token (src/shared/jwt.rs) verifica la firma HS256 con el JWT_SECRET compartido con auth-service, comprueba la expiración (exp) y el emisor (iss), y la audiencia (aud) si esta configurada.
- La verificación es local: NO se llama por red al auth-service en cada petición, lo que elimina latencia y un punto único de fallo. Basta con compartir el mismo secreto.
- Del payload se extrae el claim 'sub', que es el user_id (UUID). Ese identificador verificado es el que usa el servicio para asociar y autorizar reservas; el cliente nunca envía su user_id en el cuerpo, evitando que suplante a otro usuario.
- Cualquier fallo (firma inválida, expirado, emisor incorrecto) devuelve 401 sin revelar el motivo exacto, para no dar pistas a un atacante.

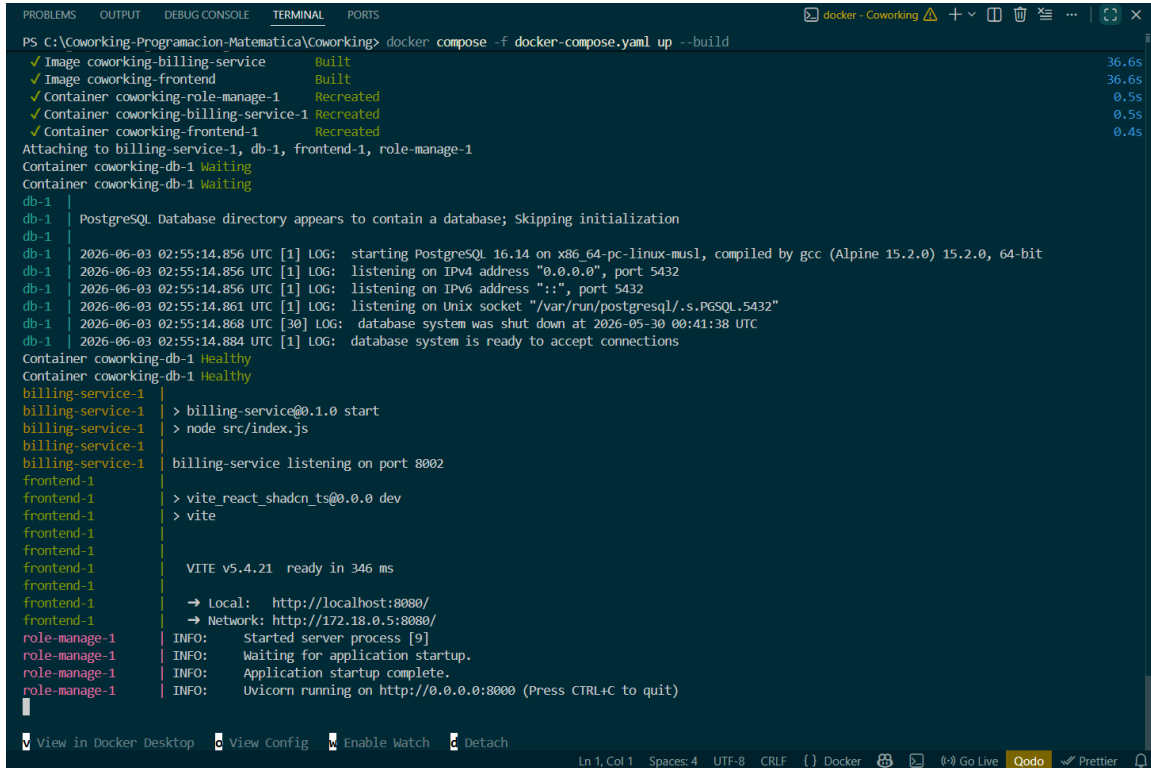
4.4 Una decisión técnica importante que tomamos

Decidimos delegar la detección de solapamiento a la base de datos con EXCLUDE USING gist en lugar de comprobarlo en código Rust con un SELECT previo seguido de un INSERT. El tradeoff: comprobarlo en código sería mas portable (no depende de PostgreSQL) y mas fácil de leer para alguien sin experiencia en SQL avanzado, pero abre una ventana de condición de carrera entre el SELECT y el INSERT donde dos peticiones concurrentes podrían ambas pasar la comprobación y crear una doble reserva.

Al ponerlo en la restricción de la base de datos, la verificación y la inserción ocurren en la misma operación atómica: es imposible que dos reservas solapadas coexistan, sin importar cuantas peticiones lleguen a la vez. El costo asumido es el acoplamiento a PostgreSQL y a sus extensiones btree_gist; lo consideramos aceptable porque la corrección ante concurrencia es la razón de ser del servicio.

A continuacion se muestran las capturas que demuestran el sistema funcionando. Las capturas no cuentan dentro del limite de 8 paginas.

Captura 1 - Docker levantando el servicio (logs de arranque)



```
PS C:\Coworking-Programacion-Matematica\Coworking> docker compose -f docker-compose.yml up --build
✓ Image coworking-billing-service Built 36.6s
✓ Image coworking-frontend Built 36.6s
✓ Container coworking-role-manage-1 Recreated 0.5s
✓ Container coworking-billing-service-1 Recreated 0.5s
✓ Container coworking-frontend-1 Recreated 0.4s
Attaching to billing-service-1, db-1, frontend-1, role-manage-1
Container coworking-db-1 Waiting
Container coworking-db-1 Waiting
db-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
db-1 |
db-1 | 2026-06-03 02:55:14.856 UTC [1] LOG: starting PostgreSQL 16.14 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
db-1 | 2026-06-03 02:55:14.856 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db-1 | 2026-06-03 02:55:14.856 UTC [1] LOG: listening on IPv6 address ":::", port 5432
db-1 | 2026-06-03 02:55:14.861 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2026-06-03 02:55:14.868 UTC [30] LOG: database system was shut down at 2026-05-30 00:41:38 UTC
db-1 | 2026-06-03 02:55:14.884 UTC [1] LOG: database system is ready to accept connections
Container coworking-db-1 Healthy
Container coworking-db-1 Healthy
billing-service-1 |
billing-service-1 | > billing-service@0.1.0 start
billing-service-1 | > node src/index.js
billing-service-1 |
billing-service-1 | billing-service listening on port 8002
frontend-1 |
frontend-1 | > vite react_shadcn_ts@0.0.0 dev
frontend-1 | > vite
frontend-1 |
frontend-1 | VITE v5.4.21 ready in 346 ms
frontend-1 |
frontend-1 | → Local: http://localhost:8080/
frontend-1 | → Network: http://172.18.0.5:8080/
role-manage-1 | INFO: Started server process [9]
role-manage-1 | INFO: Waiting for application startup.
role-manage-1 | INFO: Application startup complete.
role-manage-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Comando: `docker compose -f services/checking-service/docker-compose.yml up --build`

Captura 2 - Endpoint POST /reservations funcionando con respuesta JSON real (Postman)

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Coworking-Programacion-Matematica\Coworking> function To-Base64Url([byte[]]$bytes) {
>> [Convert]::ToBase64String($bytes).TrimEnd('=').Replace('+','-').Replace('/','_')
>> }
>>
>> function To-Base64UrlText([string]$text) {
>> To-Base64Url ([Text.Encoding]::UTF8.GetBytes($text))
>> }
>>
>> $secret = "dev-secret-change-me"
>> $userId = "550e8400-e29b-41d4-a716-446655440000"
>> $exp = [int][double]((Get-Date).ToUniversalTime().AddHours(1) - [datetime]"1970-01-01").TotalSeconds
>>
>> $header = @{ alg = "HS256"; typ = "JWT" } | ConvertTo-Json -Compress
>> $payload = @{ sub = $userId; iss = "coworking-auth"; exp = $exp } | ConvertTo-Json -Compress
>>
>> $encodedHeader = To-Base64UrlText $header
>> $encodedPayload = To-Base64UrlText $payload
>> $unsigned = "$encodedHeader.$encodedPayload"
>>
>> $hmac = New-Object System.Security.Cryptography.HMACSHA256
>> $hmac.Key = [Text.Encoding]::UTF8.GetBytes($secret)
>> $signature = To-Base64Url ($hmac.ComputeHash([Text.Encoding]::UTF8.GetBytes($unsigned)))
>>
● >> $token = "$unsigned.$signature"
PS C:\Coworking-Programacion-Matematica\Coworking> $response = Invoke-WebRequest `
>> -Uri "http://localhost:8080/reservations" `
>> -Headers @{ Authorization = "Bearer $token" } `
>> -UseBasicParsing
>>
● >> $response.Content
[]
○ PS C:\Coworking-Programacion-Matematica\Coworking>
```

Captura 3 - Health check GET /health respondiendo en el puerto 8001

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Coworking-Programacion-Matematica\Coworking> Invoke-RestMethod http://localhost:8080/health

database status
-----
up           ok

○ PS C:\Coworking-Programacion-Matematica\Coworking>
```

Respuesta esperada del health check:

```
{ "status": "ok", "db": "ok" }
```

Que fue lo mas dificil de implementar

Lo mas dificil fue conectar el modelo de intervalos del dominio Rust con el tipo TSTZRANGE de PostgreSQL respetando la semantica de fin exclusivo [inicio, fin) en ambos lados, y luego traducir el error de violacion de la restriccion EXCLUDE (SQLSTATE 23P01) a un error de negocio limpio (409 Conflict) en lugar de un 500 generico.

Que aprendimos del lenguaje o framework

Aprendimos a usar traits como puertos para invertir dependencias, de modo que la capa de aplicacion no conoce SQLx ni reqwest y puede testearse con mocks. Tambien descubrimos que PostgreSQL puede garantizar reglas de negocio de concurrencia que normalmente uno intentaria resolver en codigo, y que el extractor FromRequestParts de Axum permite exigir autenticacion por ruta con solo declarar un tipo en la firma del handler.

Que mejorariamos con mas tiempo

Con mas tiempo activariamos la lista de espera (tabla waitlist ya modelada) para que, cuando un espacio este ocupado, el usuario pueda anotarse y ser notificado al liberarse; anadiriamos un rol admin capaz de gestionar reservas de cualquier usuario; y migrariamos las consultas a las macros query! de SQLx con cache offline para validar el SQL en tiempo de compilacion.